



ACADEMIA TEHNICĂ MILITARĂ "FERDINAND I"

Facultatea de Sisteme Informatice și Securitate Cibernetică



PROIECT DE PRACTICĂ

Sistem Backup & Recovery

Nume: Std. Cap. Ungureanu Alexandru – Mirel



Cuprins

CAPITOLUL 1: INTRODUCERE ȘI FUNDAMENTE TEORETICE.....	3
1.1 Contextul securității datelor și necesitatea sistemelor de backup	3
1.2 Arhitectura sistemului și cerințe preliminare	3
CAPITOLUL 2: CONFIGURAȚIA DE SIGURANȚĂ ȘI MEDIUL DE EXECUȚIE	4
2.1 Politici stricte de execuție în Bash (Safety Flags)	4
2.2 Gestiunea resurselor temporare și prevenirea scurgerilor de date.....	5
CAPITOLUL 3: MODULUL PRINCIPAL ȘI LOGICA DE CONTROL (main.bash)	6
3.1 Analiza funcțiilor critice de arhivare	6
3.2 Logica de rutare a execuției și parserul CLI	7
CAPITOLUL 4: MODULUL DE VALIDARE ȘI STANDARDIZARE (utils.bash).....	8
4.1 Validarea datelor de intrare și a permisiunilor de acces	8
4.2 Prevenirea corupției datelor și controlul fluxului.....	9
CAPITOLUL 5: SUBSISTEMUL DE JURNALIZARE (logs.bash)	9
5.1 Mecanismul de stocare a jurnalelor și securitatea fișierelor	9
5.2 Standardizarea output-ului vizual.....	10
CAPITOLUL 6: SUBSISTEMUL DE ANALIZĂ ȘI METADATE (metadata.bash)	10
6.1 Extragerea și generarea amprentelor digitale (SHA-256).....	11
6.2 Implementarea logicii de backup diferențial.....	11
CAPITOLUL 7: IMPLEMENTAREA CRIPTĂRII ȘI RESTAURĂRII	11
7.1 Securizarea datelor cu AES-256-CBC și PBKDF2	12
7.2 Funcția de dezarhivare și decriptare a datelor (-dec).....	12
CAPITOLUL 8: AUTOMATIZAREA TESTĂRII (test.bash)	13
8.1 Structura mediului de test și scenariile validate	13
BIBLIOGRAFIE	14



CAPITOLUL 1: INTRODUCERE ȘI FUNDAMENTE TEORETICE

1.1 Contextul securității datelor și necesitatea sistemelor de backup

În peisajul tehnologic contemporan, datele reprezintă probabil cel mai valoros activ al unei organizații sau al unui utilizator individual. Incidentele cibernetice, defecțiunile hardware, erorile umane și, mai ales, atacurile de tip ransomware (care criptează datele și solicită recompense pentru decriptarea lor) au transformat simpla stocare a informațiilor într-un proces plin de riscuri. Un sistem robust de backup nu mai este un lux opțional, ci o necesitate fundamentală.

Sistemul de backup & recovery detaliat în acest proiect este un utilitar, bazat pe scripturi shell (Bash), conceput special pentru automatizarea salvării datelor. Proiectul urmărește un principiu clar: protejarea datelor nu se face doar prin simpla duplicare a acestora, ci prin adăugarea unor straturi suplimentare de validare a integrității (checksums).

1.2 Arhitectura sistemului și cerințe preliminare

Soluția propusă folosește utilitare standard prezente în majoritatea sistemelor de operare din familia UNIX/Linux. Această decizie arhitecturală maximizează portabilitatea sistemului.

Pentru a evita blocajele de execuție, mediul de operare trebuie să dispună de următoarele pachete:

- GNU Bash: Interpretorul comenzilor, responsabil de fluxul logic.
- zip / unzip: Utilitarele standard de compresie și decompresie, folosite pentru a reduce amprenta de stocare a datelor protejate.
- getfacl : Necesari pentru extragerea corectă a Listelor de Control al Accesului (Access Control Lists). Într-un mediu enterprise, permisiunile standard (Owner, Group, Other) sunt insuficiente; ACL-urile oferă granularitate, iar backup-ul trebuie să conserve aceste setări complexe.



- OpenSSL: O bibliotecă criptografică, utilizată ca motor pentru implementarea standardului de criptare AES.
- GNU Coreutils: O suită indispensabilă de unelte UNIX (stat, find, realpath, mktemp, file, sha256sum) fără de care sistemul nu ar putea efectua analiza fișierelor și manipularea căilor de pe sistem.

CAPITOLUL 2: CONFIGURAȚIA DE SIGURANȚĂ ȘI MEDIUL DE EXECUȚIE

Scripturile shell sunt notorii pentru natura lor tolerantă la erori. În mod implicit, un script Bash va continua să execute comenzi chiar dacă o comandă anterioară a eșuat sau o variabilă nu a fost declarată. Această lipsă de rigiditate poate duce la consecințe nedorite, cum ar fi ștergerea accidentală a unor directoare rădăcină (de ex. un apel de forma "rm -rf \$DIR/" unde \$DIR este neinițializat).

2.1 Politici stricte de execuție în Bash (Safety Flags)

Pentru a aduce fiabilitatea scriptului, la începutul fișierului principal "main.bash" a fost declarată directiva "set -euo pipefail". Acest construct forțează interpretorul să se comporte mai degrabă ca un limbaj compilat strict:

- Flag-ul "-e" (errexit): Instruiește shell-ul să iasă imediat din script dacă oricare comandă individuală eșuează (returnează un cod de ieșire diferit de 0). Astfel, dacă o copiere a fișierelor eșuează din lipsă de spațiu, scriptul se oprește, evitând generarea unui backup incomplet sau corupt pe care utilizatorul l-ar putea considera fals-valid.
- Flag-ul "-u" (nounset): Tratează expansiunea variabilelor neinițializate ca pe o eroare fatală, protejând sistemul împotriva typo-urilor și asigurând consistența parametrilor.
- Opțiunea "-o pipefail": În mod normal, valoarea de retur a unui pipeline (ex: "comanda_1 | comanda_2") este dată doar de ultima comandă. Dacă



activăm pipefail, Bash va captura erorile ascunse din prima parte a pipeline-ului, propagându-le corespunzător spre oprirea de urgență.

2.2 Gestiunea resurselor temporare și prevenirea scurgerilor de date

Sistemul procesează fișiere sensibile. Pentru agregarea acestora, trebuie creat un spațiu de lucru temporar. Acest spațiu este generat folosind utilitarul "mktemp -d -t backup_sys_XXXXXX", care garantează crearea unui director unic și protejat, eliminând vulnerabilitățile de tip symlink attack.

Partea vitală a managementului de memorie și securitate o reprezintă funcția de curățare ("cleanup()"). Aceasta este asociată mecanismului intern al sistemului de operare prin comanda "trap cleanup EXIT ERR INT TERM".

Indiferent de modul în care scriptul își termină execuția (cu succes, omorât prematur prin CTRL+C de utilizator, sau din pricina unei erori neprevăzute), kernelul Linux va declanșa funcția "cleanup".

Funcția verifică existența directorului temporar, și folosește "rm -rf" pentru a șterge complet orice urmă a datelor intermediare necriptate de pe disc.

CAPITOLUL 3: MODULUL PRINCIPAL ȘI LOGICA DE CONTROL (main.bash)

Fișierul main.bash reprezintă nucleul aplicației. Acesta gestionează fluxul de execuție, procesează argumentele utilizatorului și delegă sarcinile către modulele secundare. Arhitectura sa este modulară, evitând abordarea de tip "spaghetti code".

3.1 Analiza funcțiilor critice de arhivare

- Funcția `processDirectoryFiles()`: Este responsabilă de traversarea recursivă a directoarelor sursă. Pentru a rezolva eterna problemă a numelor de fișiere ce conțin spații albe sau caractere de control, funcția utilizează utilitarul `find` cu argumentul `-print0`. Acesta separă ieșirile folosind byte-ul Null (`\0`) în



loc de linie nouă. Ieșirea este apoi consumată de bucla "while IFS= read -r -d "item", garantând că absolut nicio resursă, nu va provoca o prăbușire a sistemului.

- Funcția `pushFileToTempBackupDirectory()`: Acesta este motorul individual de procesare. Funcția preia o singură cale de fișier și copiază în directorul izolat de mktemp, folosind "cp --parents". Flag-ul "--parents" este critic, deoarece păstrează exact ierarhia directorilor (ex: /home/user/docs/file.txt nu este aruncat în rădăcina backup-ului, ci este salvat sub calea relativă identică). Imediat după, funcția apelează modulul de metadata și, în cazul în care se folosește fanionul "-e", declanșează procedurile criptografice.

- Funcția `backupTmpDirectory()`: Se execută doar la finalul cu succes al tuturor iterațiilor de mai sus. După ce directorul temporar conține toate fișierele pregătite (sau criptate), comanda "zip -r" consolidează totul într-un singur container portabil trimis către locația de destinație specificată de flag-ul "-o".

3.2 Logica de rutare a execuției și parserul CLI

Interacțiunea cu sistemul este definită de un set strict de argumente procesate într-o buclă "while" pe baza poziției variabilelor "\$1" și "\$2". Funcția `checkAndPrintArguments()` asigură logica condițională și aplică un mecanism de excludere mutuală: nu se poate iniția un backup folosind simultan o listă directă ("-d") și un fișier batch ("-f").

Dicționarul complet de parametri este următorul:

- -o / --output [valoare]: Setează destinația arhivei finale. Lipsei acesteia declanșează o eroare fatală.
- -l / --log [valoare]: Solicită scrierea asincronă a evenimentelor într-un fișier text.
- -v / --verbose: Deblochează diagnosticarea pe consolă în timp real.
- -d / --directory [array]: Specifică unul sau mai multe directoare de pe partiție pentru salvare directă.



- `-f / --file [valoare]`: Permite citirea sarcinilor de lucru dintr-un fișier extern.
- `-b / --backup [valoare]`: Instruiește sistemul să facă un backup diferențial, luând ca referință o arhivă veche.
- `-e / --encrypt`: Determină intrarea datelor pe opțiunea criptografică OpenSSL.
- `-dec / --decrypt`: Inversează complet fluxul, permițând restaurarea (Restore & Recovery) datelor dintr-o arhivă precedentă.

CAPITOLUL 4: MODULUL DE VALIDARE ȘI STANDARDIZARE (utils.bash)

Sistemele de fișiere Unix se bazează puternic pe sistemul de permisiuni. Încercarea de a manipula orbește structuri de pe disc este inefficientă. Modulul `utils.bash` acționează ca un filtru de securitate.

4.1 Validarea datelor de intrare și a permisiunilor de acces

Funcția `checkFile()` validează nu doar existența unui obiect, dar forțează și validarea dreptului de citire. În sistemele multi-utilizator, dacă scriptul (rulat de utilizatorul A) încearcă să citească datele utilizatorului B fără permisiuni adecvate, nucleul sistemului de operare va refuza apelul. Verificarea manuală pre-execuție din script permite gestionarea elegantă a erorii, generând un Warning și sărind fișierul.

În mod identic, funcția `checkDirectory()` folosește testul `"-x"`. În Linux, permisiunea de "citire" pe un director îți permite doar să vezi numele fișierelor, dar permisiunea de "execuție" este obligatorie pentru a putea extrage datele din el sau a utiliza comanda `stat` pe fișierele conținute.



4.2 Prevenirea corupției datelor și controlul fluxului

Funcția `checkIfGoodZip()` este pilonul apărării în cazul restaurării. Atacatorii sau erorile umane pot re-denumi extensia unui fișier oarecare în ".zip". Dacă parserul de unzip ar încerca să citească structuri de date malformate, ar eșua imprevizibil. De aceea, scriptul folosește comanda "file -b --mime-type". Aceasta citește "numerele magice" (magic numbers) din antetul binar al fișierului, la nivel de octet, și validează dacă obiectul este, structural și criptografic, de tipul "application/zip".

Pe partea de parser CLI, funcția `checkNextMustArg()` asigură că utilizatorul nu a oferit comenzi lipsite de sens. Verifică explicit ca valoarea atribuită unui parametru să nu înceapă cu "-", blocând derapajele logice din interpretor.

CAPITOLUL 5: SUBSISTEMUL DE JURNALIZARE (logs.bash)

O aplicație de backup fără trasabilitate este un sistem "black-box" imposibil de menținut și diagnosticat în medii de producție. Jurnalizarea adecvată a tuturor metadatelor operaționale asigură capacitatea de audit pe termen lung.

5.1 Mecanismul de stocare a jurnalelor și securitatea fișierelor

Infrastructura de creare a fișierului de log este guvernată de funcția `createLogFile()`. Aceasta folosește utilitarul touch pentru a instanția un nou fișier , imediat urmat de un apel "chmod 600". Permițând doar proprietarului să citească acest fișier, mitigăm scurgerile de informații (Information Disclosure).

5.2 Standardizarea output-ului vizual

Informația de log este canalizată bidirecțional: asincron către fișier și vizual către standard output (STDOUT).

Funcția `printErrorMsg()` injectează, la interpretare, secvențe de culori (evadare ANSI) . Aceste culori, oferă o mapare vizuală extrem de rapidă în consola administratorului.



Funcția `writeLog()` folosește redirectarea fluxului de date prin operatorul ">>". Acesta este preferat operatorului ">", deoarece instruește kernelul Linux să adauge (append) informația la sfârșitul fișierului existent, menținând astfel coerența istorică și prevenind suprascrierea unor jurnale vechi.

CAPITOLUL 6: SUBSISTEMUL DE ANALIZĂ ȘI METADATE (metadata.bash)

Performanța sistemelor masive de stocare rezidă în evitarea scrierilor redundante. Pentru a atinge acest obiectiv, sistemul trebuie să implementeze o metodă irefutabilă de a distinge fișierele neschimbate de cele modificate.

Această identificare se realizează pe baza analizei metadatelor extrase de către modulul metadata.bash.

6.1 Extragerea și generarea amprentelor digitale (SHA-256).

Pentru a evalua o resursă fizică de pe partiție, funcția `extractMetaDates()` utilizează comanda "stat".

Pe lângă aceasta, se face apel la utilitarul `getfacl`. ACL-urile adaugă un layer secundar de drepturi de proprietate esențial în administrarea de nivel înalt.

Cea mai importantă componentă este validarea conținutului. Sistemul folosește funcția hash SHA-256. Orice schimbare, chiar a unui singur bit, în fișierul țintă va genera o amprentă hexazecimală complet diferită, permițând identificarea cu o acuratețe matematică de 100% a stării de modificare.\

6.2 Implementarea logicii de backup diferențial

Toate datele menționate mai sus sunt serializate într-un vector text delimitat de caracterul pipe "|" și descărcat în ".METADATA_BAK".

La declanșarea flag-ului "-b", scriptul principal despachetează metadatele din arhiva veche. Ulterior, pentru fiecare fișier procesat în iterația actuală, se generează linia curentă de metadata, care este pasată utilitarului "grep -q". Dacă comanda `grep` reușește să identifice o potrivire perfectă în string-ul bazei de date



anterioare, sistemul deduce logic că fișierul nu a suferit nicio alterare și poate fi ignorat la scriere.

CAPITOLUL 7: IMPLEMENTAREA CRIPTĂRII ȘI RESTAURĂRII

În absența unui strat de criptare integrat, fișierele ZIP extrase oferă acces instantaneu oricărui intrus. Protejarea datelor statice s-a realizat prin adăugarea suportului complet pentru utilitarul openssl.

7.1 Securizarea datelor cu AES-256-CBC și PBKDF2

Dacă sistemul depistează parametrul "-e", procedura devine una extrem de riguroasă. Funcția de protecție invocă algoritmul simetric AES. Cheia cu o lungime de 256 de biți este imposibil de spart folosind bruteforce.

O componentă crucială în configurarea OpenSSL este prelucrarea parolei introduse. Scriptul folosește directiva "-pbkdf2 -iter 100000 -salt". PBKDF2 este un standard pentru derivarea cheilor din parole. Rularea a 100.000 de iterații determină calculatorul să aplice de 100.000 de ori operațiuni matematice complexe asupra parolei înainte de a genera cheia finală, încetinind semnificativ atacatorii echipați cu cluster-e de plăci grafice care ar încerca să spargă arhiva bruteforce.

Parola însăși este extrasă prin apelul "env:BKP_PASS". Citirea parolei dintr-o variabilă de mediu a sistemului previne expunerea textului clar în istoricul terminalului sau în parametrii liniei de comandă (vizibili pentru alți useri prin comanda "ps").

7.2 Funcția de dezarhivare și decriptare a datelor (-dec)

Procesul de recuperare a datelor inversează lanțul de evenimente. Funcția `restoreAndDecrypt()` generează un director curat, prefixat cu identificatorul "RESTORED_" urmat de valoarea timestamp.



Utilizând funcția de căutare "find", scriptul vizează exclusiv extensiile ".enc", decriptând conținutul prin flag-ul "-d" (decrypt) oferit de openssl.

Procedura folosește fișiere cu prefix "tmp_dec", consolidând rezultatul pe disc (suprascrind originalul criptat) doar la confirmarea succesului procesului ("\${?} -eq 0").

CAPITOLUL 8: AUTOMATIZAREA TESTĂRII (test.bash)

Datorită multiplelor condiții de flux din interiorul parserului CLI și complexității funcțiilor I/O din main.bash, a fost creat un modul autonom de testare: test.bash.

8.1 Structura mediului de test și scenariile validate

Scriptul de test inițializează de la zero structură de "dummy files" (results/data, results/config, results/logs, results/backups) și definește variabila de mediu BKP_PASS="pass".

Cele 5 suite logice izolate pentru verificare sunt următoarele:

1. Testul T_1: Execută un proces standard de preluare de directoare utilizând doar parametrii obligați "-d" și "-o".
2. Testul T_2: Invocă funcția de criptografică, adăugând manual parametrul "-e".
3. Testul T_3: Verifică reziliența la parsare asincronă utilizând "-f". Sistemul este pus să evalueze și să proceseze linii dintr-un fișier local text cu intrări specifice.
4. Testul T_4: Simulează dinamica de utilizare în timp (Differential Testing). După ce modifică cu date noi fișierele dummy din sursă, invocă execuția cu flag-ul "-b".
5. Testul T_5: Evaluează restaurarea funcțională. Execută sistemul cu flag-ul special de rutare "-dec" peste fișierul generat anterior de



OpenSSL, validând cap-coadă algoritmul PBKDF2 și restaurând conținutul fișierelor originale în directorul de recuperare .

BIBLIOGRAFIE

1. Stallman, R., "Bash Reference Manual" - Free Software Foundation, documentație oficială (<https://www.gnu.org/software/bash/manual/>).
2. The OpenSSL Project, "OpenSSL Cryptography and SSL/TLS Toolkit" -
OpenSSL Command Line Utilities
(<https://www.openssl.org/docs/manmaster/man1/>).
3. OWASP Foundation, Cryptographic Storage Cheat Sheet (Referințe cu privire la slăbiciunile MD5 și avantajele SHA-256 și PBKDF2).
4. POSIX.1-2017 Standard - Utility Conventions and File Operations (Coreutils).